

DOCUMENTATIE.

CONSUMPTIEREGISTRATIE TVB



INHOUDSOPGAVE.

INHOUDSOPGAVE.....	2
1. DOEL VAN DE APPLICATIE.....	4
2. BESTANDEN.....	5
3. DE WEBSITE STARTEN.....	7
4. GEBRUIKERSHANDLEIDING.....	8
4.1 EEN MEDEWERKER ZOEKEN	8
4.2 EEN PRODUCT REGISTREREN	8
4.3 ADMIN-DASHBOARD OPENEN	8
4.4 REGISTRATIES BEHEREN	8
4.5 MEDEWERKERS BEHEREN	9
4.6 PRODUCTEN BEHEREN	10
4.7 BEDRIJVEN EN CONSUMPTIEPUNTEN BEHEREN	10
4.8 VOORRAAD BEHEREN	10
4.9 LOGBOEK BEKIJKEN	11
4.10 ALLE GEGEVENS WISSEN.....	11
4.11 MODAL SLUITEN EN UITLOGGEN.....	12
4.12 CSV EXPORTEREN	12
4.13 LICHT OF DONKER THEMA	12
4.14 WELKOM EN "ZELFDE ALS VORIGE KEER"	13
4.15 DEMO GEZICHTSHERKENNING	13
5. UITLEG VAN <code>index.html</code>	14
5.1 METADATA EN EXTERNE BESTANDEN	14
5.2 NAVIGATIEBALK.....	14
5.3 HERO-GEDEELTE	14
5.4 MEDEWERKERLIJST	14
5.5 STATISTIEKEN	15
5.6 VENSTERS (MODALS)	15
5.7 WAARSCHUWING BIJ OPENEN ALS LOS BESTAND	15
6. UITLEG VAN <code>assets/css/styles.css</code>	16
6.1 CSS-VARIABELEN	16
6.2 LAYOUT	16
6.3 COMPONENTEN	17
7. UITLEG VAN DE JAVASCRIPT (<code>assets/js/</code>).....	18
7.1 OPSLAG EN VOORBEELDDATA.....	18
7.2 OBJECTGEORIËNTEERDE STRUCTUUR	19
7.3 <code>renderEmployees</code>	22
7.4 <code>renderStats</code>	22

7.5	<code>registerSelectedProducts</code>	23
7.6	<code>renderAll</code>	23
7.7	<code>renderEmployeeProducts</code>	23
7.8	<code>showToast</code>	23
7.9	OPSLAGFOUTEN EN TERUGDRAAIEN	23
7.10	ADMINFUNCTIES.....	23
7.11	CSV-EXPORT (CLASS <code>CsvExport</code> IN <code>csvExport.js</code>).....	24
7.12	EVENT LISTENERS.....	24
8.	DATAMODEL EN KOPPELINGEN.....	26
8.1	BEDRIJVEN.....	26
8.2	CONSUMPTIEPUNTEN	26
8.3	MEDEWERKERS.....	26
8.4	REGISTRATIES.....	26
9.	BELANGRIJKE DEMO-BEPERKINGEN.....	28
10.	CODE UITLEGGEN TIJDENS EEN PRESENTATIE.....	29
11.	MOGELIJKE VERVOLGSTAPPEN.....	30

1. DOEL VAN DE APPLICATIE.

Consumptieregistratie is een eenvoudige website waarmee medewerkers eten en drinken kunnen registreren. De standaardproducten zijn blikje (€0,65), sneetje brood (€0,10), boter (€0,10), zoet beleg (€0,20), glas melk (€0,20), beleg (€0,50), ei (€0,50) en yoghurt (€0,50). Een medewerker kiest de producten met + en - en slaat ze op met Registreren; ieder gekozen product wordt dan één registratie. Persoonlijke aantallen en kosten zijn alleen zichtbaar voor de beheerder.

Deze versie is een **frontend-demo**. De gegevens worden opgeslagen in `localStorage` van de gebruikte browser. Daardoor zijn de gegevens op één computer/browser beschikbaar, maar nog niet gedeeld tussen alle gebruikers van een bedrijfsnetwerk.

2. BESTANDEN.

Bestand	Functie
index.html	De HTML-structuur en alle zichtbare onderdelen van de website
assets/css/styles.css	De volledige vormgeving, kleuren, layout en mobiele weergave
assets/js/main.js	Startpunt: maakt opslag, model, view en controller aan en start de app
assets/js/config.js	Vaste waarden: opslagsleutel, kleuren, voorbeeldmedewerkers, standaardproducten, de 13 bedrijven, de tijden van zonsopkomst en zonsondergang en de instellingen van de demo gezichtsherkenning (FACE_DEMO)
assets/js/DataStore.js	Opslag in <code>localStorage</code> , validatie en omzetting van oude gegevens
assets/js/RegistrationModel.js	Gegevens en regels voor registraties, medewerkers, producten en logboek
assets/js/RegistrationView.js	Weergave van alle onderdelen als HTML
assets/js/RegistrationApp.js	Controller: verwerkt klikken, formulieren en toetsenbordacties
assets/js/csvExport.js	Class <code>CsvExport</code> : CSV-export voor de loonadministratie (maand + 1)
assets/js/ThemeManager.js	Class <code>ThemeManager</code> : licht en donker thema (Systeem, Auto of zelf kiezen met de slider)
assets/js/icons.js	De SVG-iconen van de website, met de hulpfunctie <code>icon(naam)</code>
assets/js/ids.js	De functie <code>createId()</code> voor unieke id's; werkt ook als de website via een netwerkadres zonder https wordt geopend
assets/js/FaceRecognitionDemo.js	Class <code>FaceRecognitionDemo</code> : de uitschakelbare demo gezichtsherkenning (zie 4.15)
assets/fonts/	De lettertypes DM Sans en Space Grotesk (woff2), zelf gehost in plaats van via Google Fonts. Licentie: SIL Open Font License 1.1 (LICENSE.txt)
assets/vendor/face-api/	De bibliotheek face-api (<code>face-api.esm.js</code> , versie 1.7.15) en de drie modellen voor de demo gezichtsherkenning, met de MIT-licentie (LICENSE). Samen ongeveer 8 MB, waarvan 6,4 MB het herkenningsmodel. Wordt alleen geladen als iemand de demo gebruikt. <code>README.md</code> in deze map noemt de bron en een SHA-256-controlegetal per bestand
archief/app.js	Back-up van de originele versie in één bestand; wordt niet meer geladen
archief/app.test.js	Back-up van de oude tests bij die versie; wordt niet meer uitgevoerd
README.md	Korte startinformatie voor het project
tools/build-docs.ps1 en tools/md-to-docx.cjs	Maken van <code>docs/TO-BLIKJESREGISTRATIE.md</code> en <code>docs/DOCUMENTATIE.md</code> een Word-bestand (<code>.docx</code>) en een PDF in de TVB-huisstijl, op basis van het sjabloon <code>docs/tvbsjabloon.docx</code> : voorblad, inhoudsopgave, genummerde koppen, tabellen en opsommingstekens. Nodig: Node.js en Microsoft Word. De <code>.md</code> -bestanden blijven de bron: een wijziging die alleen in de <code>.docx</code> staat, is weg na de volgende keer maken
docs/	Technisch ontwerp (met daarin ook de functionele eisen en de wireframes), databaseschema en deze documentatie
docs/wireframes/	Low-fidelity (lofi-...), mid-fidelity (midfi-...) en high-fidelity (w...) wireframes van de schermen, gebruikt in het TO (hoofdstuk 9)
tests/	Unit tests (480) voor de regels, de opslag, de export, het thema, de welkom- en gezichtsherkenningfuncties, de controller, de beveiliging en regressietests voor opgeloste bugs

Bestand	Functie
tests/opslag-model-export.test.js	Tests voor de reservekopieën, het wissen van gegevens, de strengere controle bij het laden, de voorraad telling (countedAt), nieuwe modelmethodes en de beveiliging van de CSV-export, verboden id's, maximale tekstlengtes en een opslag die niet te lezen is
tests/controller-robustheid.test.js	Tests voor de robuustheid van controller en view: na middernacht, correcties in een oude maand, logboek en wijziging in één keer opslaan, grenzen bij de voorraad, verouderde formulieren en knoppen, de camera, de focus na sluiten, veilige HTML, Alle gegevens wissen, en exporteren en wissen alleen voor een ingelogde beheerder
tests/beveiliging-config.test.js	Tests voor de configuratie: geen externe bronnen, de Content-Security-Policy, lokale lettertypes en face-api (met de SHA-256-controlegetallen), en de CI-workflow
tests/gegevens-en-tabbladen.test.js	Tests voor de prijsgrens van 1000 euro, de focus na Registreren, "niets gewijzigd" bij opslaan, en verouderde formulieren of een gewiste opslag na een wijziging in een ander tabblad. Gebruikt de strenge nep-view (createStrictView in helpers.js)
tests/werkgever-en-correctiedatum.test.js	Tests voor het werkgevern timer per registratie en de datum bij een correctie +
tests/to-controle.test.js	Regressietests voor de TO-controle: "1 medewerker"/"2 medewerkers", bevestiging bij product verwijderen, de melding bij een verwijderde medewerker (correctie +), alleen een inactieve medewerker zonder registraties verwijderen, uniek werkgevern timer, dubbele id's en datums in de toekomst bij het laden, de apostrof in de CSV, lastRegistration en pointId, het SQL-schema (counted_at, geen rol manager), de opslagknoppen bij wijzigen, de filters op een smal scherm en tests voor FR-01, FR-02, FR-04, FR-10 en FR-29 (actieve medewerkers, zoeken op naam, de - in het productvenster, alle registraties voor de beheerder, geen bedragen of aantallen op de publieke pagina). Verder: "1 product" in de correctielijst, een klik op de medewerkerkaart (FR-32) en het totaal op de knop Registreren (FR-47)
.gitattributes	Zorgt dat Git de bestanden in assets/vendor/ en assets/fonts/ niet aanpast (geen omzetting van regeleinden), zodat de controlegetallen blijven kloppen
.github/workflows/ci.yml	Voert de unit tests automatisch uit op GitHub, met alleen leesrechten en actions die op een commit-SHA zijn vastgezet

Bij het toevoegen of wijzigen van een medewerker kiest de beheerder een bedrijf uit de keuzelijst en daarna een consumptiepunt van dat bedrijf. Met het bedrijf worden medewerkers op de beginpagina gegroepeerd en gefilterd. Het bedrijfsfilter op de beginpagina is een keuzelijst waarin je ook kunt typen; de lijst filtert direct tijdens het typen en toont alleen bedrijven met actieve medewerkers. Wat je typt, wordt niet overschreven als de lijst intussen opnieuw wordt getekend. Hebben actieve medewerkers geen bedrijf, of bestaat hun bedrijf niet meer, dan staat er ook de keuze `onbekend bedrijf`; die toont al deze medewerkers.

3. DE WEBSITE STARTEN.

1. Open de projectmap in VS Code.
2. Klik met de rechtermuisknop op `index.html` en kies **Open with Live Server** (extensie "Live Server"). Een andere lokale webserver werkt ook.
3. De website opent in een moderne browser.
4. Er is geen installatie of buildproces nodig.

Alle bestanden staan in het project zelf: ook de lettertypes (`assets/fonts/`) en de bibliotheek en modellen van de demo gezichtsherkenning (`assets/vendor/face-api/`). De website laadt dus niets van een externe server en werkt zonder internetverbinding, zolang de lokale webserver draait.

De website werkt ook via een netwerkadres, bijvoorbeeld `http://192.168.1.20:5500` op een tablet in hetzelfde netwerk. Alleen de demo gezichtsherkenning is dan niet beschikbaar, omdat browsers de camera alleen via `https://` of `http://localhost` toestaan.

Dubbelklikken op `index.html` werkt niet: de JavaScript is opgedeeld in modules, en browsers blokkeren modules die via `file://` worden geopend. De pagina toont dan een waarschuwing.

4. GEBRUIKERSHANDLEIDING.

4.1 EEN MEDEWERKER ZOEKEN

Gebruik de zoekbalk boven de medewerkerlijst. Typ een voornaam, achternaam of een deel van een naam. De lijst wordt direct gefilterd.

Met **Ctrl + K** op Windows of **Cmd + K** op macOS krijgt de zoekbalk automatisch de focus. Dat werkt alleen als er geen venster open is, omdat de zoekbalk dan achter het venster staat.

De datum bovenaan en de totalen van vandaag en deze maand worden na middernacht vanzelf bijgewerkt (binnen een minuut). De tablet bij het consumptiepunt kan dus dag en nacht aan blijven staan.

Sluit je een venster (met het kruisje, een klik naast het venster, Escape of na opslaan), dan gaat de focus terug naar de knop of medewerkerrij waarmee je het venster opende. Met het toetsenbord kun je dan meteen verder waar je was.

4.2 EEN PRODUCT REGISTREREN

1. Zoek je eigen naam.
2. Klik op je naam. Het productvenster opent met alleen de producten van jouw consumptiepunt.
3. Klik bij ieder gewenst product op **+**. Met **-** haal je een verkeerde keuze weer weg (niet lager dan 0).
4. Klik op Registreren (aantal) om alle gekozen producten tegelijk op te slaan.
5. Onderaan verschijnt 4 seconden een melding met je naam en de producten, bijvoorbeeld Lotte van Dijk: 2x Blikje, 1x Ei geregistreerd.

Iedere tik op **+** telt als één product. De datum en tijd worden automatisch toegevoegd zodra je op Registreren klikt. Je ziet geen persoonlijk totaal of persoonlijke kosten. Klik je op Registreren zonder iets te kiezen, dan verschijnt "Kies eerst minimaal één product". Biedt je consumptiepunt nog niets aan, dan staat er "Geen producten op jouw consumptiepunt."

4.3 ADMIN-DASHBOARD OPENEN

1. Klik rechtsboven op Admin login.
2. Vul een e-mailadres in.
3. Vul een wachtwoord in.
4. Klik op Inloggen.

In deze demo wordt niet gecontroleerd of het wachtwoord echt correct is. Elk ingevuld wachtwoord werkt. Dit is alleen geschikt voor demonstratie en niet voor productie.

4.4 REGISTRATIES BEHEREN

In het tabblad Registraties ziet de beheerder:

- De filters Medewerker en Maand en de knop CSV exporteren
- Onder Medewerker corrigeren: een zoekveld en per medewerker het aantal producten ("1 product", "2 producten") en het bedrag
- Een tabel met medewerker, product, aantal en datum en tijd, nieuwste bovenaan (bij een registratie uit een ander jaar staat het jaartal erbij)

De beheerder zoekt onder **Medewerker corrigeren** eerst een medewerker op en klappt de rij open. Per product staan daar een rode -, het aantal en een groene +. Met + wordt één registratie van dat product toegevoegd, op de datum in het veld **Datum bij toevoegen (+)**. Dat veld staat na het inloggen op vandaag. Kies een eerdere datum voor een vergeten registratie; een datum in de toekomst geeft een foutmelding. Valt de gekozen datum in een eerdere maand, dan vraagt de browser eerst: "Deze registratie komt in september 2026. Die loonmaand is mogelijk al verwerkt. Toch toevoegen?". De datum komt dan ook in het logboek en in de melding ("... op 14 september 2026"). Met - wordt de laatst ingevoerde registratie van dat product bij die medewerker verwijderd (de volgorde van invoer telt, niet de datum: een correctie + met een eerdere datum die als laatste is toegevoegd, gaat dus als eerste weg). Is die registratie uit een andere maand dan de huidige, dan vraagt de browser eerst: "Deze registratie is van september 2026. Die loonmaand is mogelijk al verwerkt. Toch verwijderen?" (met de maand van de registratie). Klik alleen op OK als je zeker weet dat de loonadministratie die maand nog niet heeft verwerkt; met **Annuleren** verandert er niets. Heeft de medewerker geen registratie van dat product, dan verschijnt de foutmelding "Dit product heeft geen registratie voor deze medewerker". Iedere correctie komt in het logboek; de correctie en de logboekregel worden samen opgeslagen. Een correctie met + telt niet mee als "Vorige keer koos je ..." in het productvenster van de medewerker, omdat de medewerker die keuze niet zelf heeft gemaakt. De losse registraties in de tabel zijn alleen ter inzage; correcties gebeuren dus altijd via **Medewerker corrigeren**.

De keuzelijsten boven de tabel filteren op medewerker of maand. Zijn er voor de filters geen registraties, dan staat er "Geen registraties voor deze filters."

4.5 MEDEWERKERS BEHEREN

Open het tabblad **Medewerkers**.

- ⬢ Vul voornaam, achternaam, looncode en personeelsnummer in.
- ⬢ Kies het bedrijf en daarna het vaste consumptiepunt van de medewerker. De keuzelijst toont alleen de punten van het gekozen bedrijf. De medewerker ziet bij het registreren alleen de producten van dat punt, en de voorraad van dat punt gaat omlaag.
- ⬢ Het werkgevernnummer komt standaard van het bedrijf. Heeft een medewerker een ander werkgevernnummer (bijvoorbeeld per teamleider), vul dan **Afwijkend werkgevernnummer** in; dat nummer gaat dan voor in de CSV-export.
- ⬢ Klik op **Opslaan** om een medewerker toe te voegen. **Opslaan + opnieuw** houdt het bedrijf en consumptiepunt vast voor de volgende collega.
- ⬢ Gebruik **Wijzigen** om medewerkergegevens aan te passen.
- ⬢ Gebruik **Deactiveren** of **Activeren** om de medewerkerstatus te wijzigen.

Voornaam, achternaam, looncode, personeelsnummer en bedrijf zijn verplicht, en ook het consumptiepunt als het gekozen bedrijf consumptiepunten heeft. Ontbreekt er iets, dan krijgt het veld een rode rand met een melding eronder en krijgt het eerste foute veld de focus. Looncode, personeelsnummer en een afwijkend werkgevernnummer mogen alleen cijfers bevatten; anders verschijnt "Gebruik alleen cijfers." bij het veld. De voornaam en achternaam worden apart opgeslagen (`firstName`, `lastName`); de volledige naam (`name`) wordt daaruit samengesteld.

Een inactieve medewerker zonder registraties kan daarna met **Verwijderen** definitief worden verwijderd; de browser vraagt eerst om bevestiging. Heeft de medewerker registraties, dan staat de knop er niet en is deactiveren voldoende: zo blijven naam, looncode en personeelsnummer beschikbaar voor de CSV-export. Ook bij een actieve medewerker staat de knop er niet. Controller en model controleren dit zelf ook, voor het geval een knop verouderd is (bijvoorbeeld na een wijziging in een ander tabblad): bij registraties verschijnt "Deze medewerker heeft registraties en kan niet definitief worden verwijderd. Deactiveren is

voldoende.", bij een actieve medewerker "Zet deze medewerker eerst op inactief. Alleen een inactieve medewerker kan definitief worden verwijderd." Een personeelsnummer mag maar bij één medewerker voorkomen.

4.6 PRODUCTEN BEHEREN

Open het tabblad **Producten** en klik op **+ Product toevoegen**. Het formulier heeft dezelfde opties als het medewerkersformulier:

- Opslaan: opslaan en het formulier sluiten.
- Opslaan + opnieuw: opslaan, de velden leegmaken en het formulier open laten.

Een nieuw product staat bij alle consumptiepunten **uit**. Zet het daarna aan bij de punten die het aanbieden (zie 4.7).

De prijs is 0 of hoger en heeft hooguit twee decimalen (hele centen), bijvoorbeeld 0.65; anders verschijnt "Vul een prijs van 0 of hoger in, met hooguit twee decimalen." Twee producten met dezelfde naam kan niet: dan verschijnt "Er bestaat al een product met deze naam." Een product dat al eens is geregistreerd, kan niet worden verwijderd, zodat oude registraties en de export blijven kloppen ("Dit product wordt al gebruikt en kan niet worden verwijderd"). Bij een product dat nooit is geregistreerd, vraagt de browser eerst "Product Boter verwijderen?" (met de naam van het product); met **Annuleren** verandert er niets.

4.7 BEDRIJVEN EN CONSUMPTIEPUNTEN BEHEREN

Open het tabblad **Bedrijven**. Standaard staan de 13 bedrijven van TVB erin. Onder ieder bedrijf staan de consumptiepunten van dat bedrijf. Een bedrijf kan geen, één of meerdere consumptiepunten hebben.

- + Bedrijf toevoegen** en **Wijzigen**: bedrijfsnaam en werkgevernnummer invullen. Het werkgevernnummer komt in de CSV-export bij de medewerkers van dat bedrijf, behalve bij medewerkers met een afwijkend werkgevernnummer. Bedrijfsnaam en werkgevernnummer zijn uniek: bij een dubbele waarde verschijnt bij het veld "Er bestaat al een bedrijf met deze naam." of "Er bestaat al een bedrijf met dit werkgevernnummer." en wordt er niets opgeslagen. Meerdere bedrijven zonder werkgevernnummer mag wel. Dit past bij de UNIQUE-regel in `docs/DATABASE-SCHEMA.sql`.
- Bij ieder bedrijf en consumptiepunt staat het aantal gekoppelde medewerkers, in enkelvoud of meervoud ("1 medewerker", "2 medewerkers").
- + Consumptiepunt**: een nieuw punt bij dat bedrijf toevoegen. Geef het een naam (bijv. **Kantine begane grond**) en vink aan welke producten er worden aangeboden.
- Aanbod wijzigen**: naam, bedrijf of aangeboden producten van een punt aanpassen. De voorraad blijft bewaard, ook als een product tijdelijk uit staat. Kies je een ander bedrijf, dan verhuizen de medewerkers van dit punt mee naar dat bedrijf; alleen hun bedrijf verandert, hun naam en andere gegevens blijven gelijk.
- Verwijderen**: kan alleen als er geen medewerkers meer aan het bedrijf of punt gekoppeld zijn. Een bedrijf kan ook pas worden verwijderd als het geen consumptiepunten meer heeft. De browser vraagt eerst om bevestiging.

4.8 VOORRAAD BEHEREN

Open het tabblad **Voorraad**.

- Bovenaan staat de lijst **Bijbestellen** met alle aangeboden producten die op zijn (voorraad 0 of minder) of op of onder het minimum zitten (**Bijbestellen**), over alle consumptiepunten. Klik op een punt om de voorraad daarvan te openen.

- ⬢ Kies een consumptiepunt om de voorraad per product te zien.
- ⬢ **Levering:** vul het geleverde aantal in (een heel getal van 1 tot en met 100.000) en klik op + **Toevoegen**. Het aantal komt bij de voorraad. Tik je per ongeluk twee keer snel achter elkaar (binnen 800 milliseconden) op dezelfde knop, dan telt alleen de eerste tik.
- ⬢ **Voorraad:** na het tellen kun je het getal direct aanpassen (een heel getal van 0 tot en met 100.000). Het wordt opgeslagen zodra je het veld verlaat. Het systeem onthoudt dan ook het tijdstip van de telling (**countedAt**).
- ⬢ **Minimum:** bij dit aantal of minder verschijnt het product in de bijbestellijst (een heel getal van 0 tot en met 100.000).

Iedere registratie haalt automatisch 1 van de voorraad af bij het consumptiepunt van de medewerker, maar alleen als dat punt het product aanbiedt (een correctie voor een product dat het punt niet heeft, verandert de voorraad dus niet). Na het aanpassen van voorraad of minimum worden alleen het statuslabel en de bijbestellijst bijgewerkt; de tabel blijft staan, zodat je met Tab gewoon naar het volgende veld kunt. Een foute invoer (leeg, geen heel getal, negatief of meer dan 100.000) geeft een melding en springt terug naar de opgeslagen waarde.

Een correctie met + op een datum vóór de laatste telling haalt niets van de voorraad af: dat product zat toen al niet meer in de getelde hoeveelheid. Een correctie met - in het admin-dashboard zet het product weer terug in de voorraad, maar alleen als de registratie ná de laatste telling is gemaakt. Is de voorraad daarna geteld, dan zat het product al niet meer in de getelde hoeveelheid en blijft de telling staan.

Een negatieve telling invullen kan niet. Door registraties kan de voorraad wel onder 0 komen; dat betekent dat er meer is geregistreerd dan er volgens de telling was. Tel dan opnieuw. Alle leveringen, tellingen en wijzigingen van het minimum komen in het logboek.

Typ je in de voorraadtabel terwijl in een ander tabblad iets wordt opgeslagen, dan blijft de tabel staan, zodat je half ingetypte getal niet verdwijnt.

4.9 LOGBOEK BEKIJKEN

Het tabblad **Logboek** toont standaard maximaal 20 wijzigingen. Met **Meer laden** worden steeds 20 extra wijzigingen getoond.

4.10 ALLE GEGEVENS WISSEN

Onder het logboek staat de rode knop **Alle gegevens wissen**. Daarmee wis je alle persoonsgegevens op de tablet, bijvoorbeeld als de tablet ergens anders wordt gebruikt (recht op vergetelheid uit de AVG).

1. Klik op **Alle gegevens wissen**.
2. De browser vraagt: "Alle medewerkers, registraties, producten, voorraad, het logboek en de reservekopieën op deze tablet wissen?" Klik op OK.
3. De browser vraagt nog een keer: "Weet je het zeker? Dit kan niet ongedaan worden gemaakt." Klik op OK.

Daarna zijn alle gegevens, beide reservekopieën en de ingestelde gezichten van de demo gewist. Je bent uitgelogd, het beheervenster is dicht en de demogegevens staan er weer (8 voorbeeldmedewerkers, de standaardproducten, de 13 bedrijven en het consumptiepunt Hoofdkantoor). Er verschijnt de melding "Alle gegevens zijn gewist. De demogegevens staan er weer." Klik je bij een van de twee vragen op **Annuleren**, dan verandert er niets. Andere open tabbladen laden de nieuwe demogegevens vanzelf in.

4.11 MODAL SLUITEN EN UITLOGGEN

Het kruisje rechtsboven, een klik naast het venster of de Escape-toets sluit het admin-overzicht én logt de beheerder uit. Op een gedeelde tablet kan de volgende persoon zo niet zonder wachtwoord in het beheer komen. Bij het volgende openen verschijnt het loginformulier. Met de knop `Uitloggen` kan de beheerder ook uitloggen zonder het venster te sluiten.

4.12 CSV EXPORTEREN

1. Open het admin-dashboard.
2. Kies eventueel een medewerker en/of maand. Achter iedere maand staat de loonmaand waarin die wordt verwerkt, bijvoorbeeld `september 2026` (loonmaand oktober 2026).
3. Klik op `CSV exporteren`.
4. De browser downloadt een `.csv`-bestand, bijvoorbeeld `blikjesregistratie-loonmaand-2026-10.csv`.

Het bestand heeft altijd de kolommen `Jaar`, `Maand`, `Looncode`, `Personeelsnummer`, `Werkgevernummer`, `Naam`, `Totaal` en `Prijs`, in die vaste volgorde. Iedere regel is één medewerker in één loonmaand. Het werkgevernummer is het nummer dat bij de registraties is bewaard (zie de tabel Registraties hieronder); is dat binnen een maand veranderd, dan krijgt die maand één regel per werkgevernummer. Er staat bewust geen totaalregel in, omdat de loonadministratie iedere regel als medewerker inleest.

Loonmaand: consumpties worden verwerkt in de loonadministratie van de maand erna. `Jaar` en `Maand` in de export zijn daarom de loonmaand: consumpties uit september 2026 staan als `2026 / 10` in het bestand, consumpties uit december 2026 als `2027 / 1`.

Het bestand gebruikt puntkomma's als scheidingsteken en een komma als decimaalteken (`0,65`), zodat een Nederlandse Excel het direct goed opent. De gekozen filters worden toegepast op de export. De prijs komt van iedere registratie zelf: dat is de prijs op het moment van registreren. Verandert een prijs later, dan blijft een oude maand in de export hetzelfde. Alleen oude registraties zonder opgeslagen prijs gebruiken de huidige prijs. Losse registraties staan in de tabel in het admin-dashboard.

4.13 LICHT OF DONKER THEMA

Rechtsboven in de bovenbalk staat de thema-schakelaar:

- 🔘 **Systeem** (standaard): de website volgt de instelling van je computer of telefoon. Staat Windows of je telefoon op donker, dan is de website ook donker.
- 🔘 **Slider (met zon- en maanicoon)**: kies zelf licht of donker. Dit is een vaste keuze; "Systeem" en "Auto" staan daarna uit.
- 🔘 **Auto**: overdag licht en na zonsondergang donker. De tijden van zonsopkomst en zonsondergang zijn per maand ingesteld voor Nederland (`DAYLIGHT_HOURS` in `config.js`), bijvoorbeeld in december donker vanaf ongeveer 16:30 en in juni vanaf ongeveer 22:00. De website controleert iedere minuut of het tijd is om te wisselen.

De keuze wordt per browser onthouden. In het donkere thema blijven de huiskleuren behouden: de achtergrond wordt donker en de tekst krijgt de groene huiskleur, iets lichter zodat hij goed leesbaar is.

4.14 WELKOM EN "ZELFDE ALS VORIGE KEER"

Als een medewerker het productvenster opent, verschijnt een persoonlijke begroeting die past bij het tijdstip, bijvoorbeeld "Goedemorgen Lotte, welkom terug." Heeft de medewerker eerder iets geregistreerd, dan staat erboven wat er de vorige keer is gekozen ("Vorige keer koos je 1× Blikje en 1× Ei."). Met de knop `Zelfde als vorige keer` staan die aantallen direct klaar; de medewerker kan ze nog aanpassen en bevestigt met `Registreren`. Correcties van de beheerder tellen hierbij niet mee, ook niet voor "welkom terug". Bij het openen staat de focus op de eerste `+`-knop; na een druk op `+` of `-` houdt dezelfde knop de focus, zodat je met het toetsenbord meerdere keren kunt drukken.

4.15 DEMO GEZICHTSHERKENNING

Dit is een demo; zie het TO (hoofdstuk 13, Privacy) waarom gezichtsherkenning niet geschikt is voor de echte toepassing.

1. Kies je naam in de lijst. Onderaan het productvenster staat `Gezichtsherkenning instellen (demo)`.
2. De camera van de tablet of laptop gaat aan. Plaats je gezicht in het kader, zet het vinkje voor vrijwillige deelname en klik op `Gezicht vastleggen`.
3. Voortaan kun je op de beginpagina op `Herken mij met de camera (demo)` klikken. Word je herkend, dan opent je eigen productvenster.

Er worden geen foto's gemaakt of opgeslagen; de herkenning gebeurt op het apparaat zelf. Na het herladen van de pagina is niemand meer ingesteld. Met `Uitzetten` in het productvenster vergeet de demo je gezicht direct; `Alle gegevens wissen` vergeet alle ingestelde gezichten. De bibliotheek en de modellen komen uit het project zelf (`assets/vendor/face-api/`), niet van internet. Met `FACE_DEMO.enabled = false` in `config.js` staat de demo helemaal uit. De camera werkt alleen via `http://localhost` of `https://`.

5. UITLEG VAN `index.html`.

5.1 METADATA EN EXTERNE BESTANDEN

In de `<head>` staan:

- ⬢ `lang="nl"`: de pagina is Nederlandstalig.
- ⬢ `charset="utf-8"`: ondersteunt Nederlandse tekens.
- ⬢ `viewport`: maakt de pagina geschikt voor mobiel.
- ⬢ De titel en beschrijving voor browser en zoekmachines.
- ⬢ `referrer` op `no-referrer`: de browser stuurt bij links en verzoeken het adres van de pagina niet mee.
- ⬢ Een Content-Security-Policy (`http-equiv="Content-Security-Policy"`): de browser laadt alleen bestanden van de eigen server. De twee inline scripts (het themascript in de `<head>` en de `file://`-waarschuwing onderaan) zijn toegestaan via hun sha256-hash. Wijzig je zo'n script, dan moet de hash in de CSP mee veranderen; `tests/beveiliging-config.test.js` controleert dat. `frame-ancestors` werkt niet in een meta-tag; die moet in productie als HTTP-header door de webserver worden meegestuurd (zie het TO, hoofdstuk 13).
- ⬢ `assets/css/styles.css` voor de vormgeving. De lettertypes worden daar met `@font-face` uit `assets/fonts/` geladen, niet van Google Fonts.
- ⬢ `assets/js/main.js` als startpunt van de JavaScript.

5.2 NAVIGATIEBALK

De `.topbar` bevat:

- ⬢ Het TVB-logo.
- ⬢ De naam van de applicatie.
- ⬢ De themakeuze: een slider voor licht of donker en de knoppen `Systeem` en `Auto` (zie 4.13).
- ⬢ De status `Systeem actief`.
- ⬢ De knop `Admin login` waarmee het admin-dashboard wordt geopend.

De HTML is ingedeeld in herkenbare componentblokken: `header` voor navigatie, `main` voor de hoofdinhoud, `section` voor inhoudelijke onderdelen, `aside` voor samenvattingen en `footer` voor de afsluiting. De admin-modal bevat afzonderlijke componenten voor login, tabs, filters en beheeracties. JavaScript gebruikt de `id`-waarden als koppeling met deze componenten.

5.3 HERO-GEDEELTE

De `.hero` toont de datum van vandaag, een introductietekst en een decoratief recycling-symbool.

De datum wordt niet hardcoded ingevuld. JavaScript vult het element met id `todayLabel` automatisch.

5.4 MEDEWERKERLIJST

De `.directory-card` bevat:

- ⬢ De titel `Registreer je consumptie`.
- ⬢ De teller met het aantal medewerkers (id `employeeCount`).

- De knop Herken mij met de camera (demo) (id `faceRecognizeButton`); alleen zichtbaar als de demo aan staat en de browser een camera kan gebruiken.
- De zoekbalk met id `employeeSearch` en de sneltoets-hint `Ctrl K`.
- Het bedrijfsfilter met id `employeeCompanyFilter` en de keuzelijst `companyFilterOptions`.
- De lege status met id `emptyState`, met de knop Zoekopdracht wissen.
- De dynamische lijst met id `employeeList`.

JavaScript vult `employeeList` op basis van de medewerkers in de applicatiestatus.

5.5 STATISTIEKEN

De elementen `todayTotal` en `monthTotal` worden door JavaScript gevuld met:

- Het aantal registraties van vandaag.
- Het aantal registraties van de huidige maand.

Daaronder staat de tipkaart `Goed bezig!`.

5.6 VENSTERS (MODALS)

Ieder venster is een `.modal-backdrop` over de website heen. Een venster sluit met het kruisje, met een klik naast het venster of met de Escape-toets (dan sluit het bovenste venster). Met Tab blijft de focus binnen het open venster. Bij het openen staat de focus meteen in het venster, en het admin-venster heeft altijd de naam van de zichtbare kop ("Welkom terug" of "Overzicht"). Na het sluiten gaat de focus terug naar het element waarmee het venster werd geopend.

Id	Venster
<code>adminModal</code>	Loginformulier (<code>loginView</code>) en beheerdashboard (<code>adminView</code>) met zes tabbladen
<code>employeeProductsModal</code>	Productvenster van een medewerker, met begroeting, voorstel Zelfde als vorige keer en de link voor de demo gezichtsherkenning
<code>employeeFormModal</code>	Medewerker toevoegen of wijzigen
<code>productFormModal</code>	Product toevoegen of wijzigen
<code>companyFormModal</code>	Bedrijf toevoegen of wijzigen
<code>pointFormModal</code>	Consumptiepunt toevoegen of het aanbod wijzigen
<code>faceModal</code>	Camerabeeld van de demo gezichtsherkenning (instellen of herkennen)

Het element `toast` toont de korte meldingen onderaan het scherm.

De knop met `data-close-modal` sluit het admin-venster en logt de beheerder daarbij ook uit (zie 4.11). De knop `Uitloggen` logt uit zonder het venster te sluiten; daarna staat het loginformulier in hetzelfde venster.

Onder het logboek (`auditTab`) staan een uitleg en de knop `Alle gegevens wissen` (id `wipeDataButton`, zie 4.10).

5.7 WAARSCHUWING BIJ OPENEN ALS LOS BESTAND

Onderaan `index.html` staat een klein script dat controleert of de pagina via `file://` is geopend. Is dat zo, dan verschijnt bovenaan een balk met de uitleg dat de website via Live Server moet worden gestart (zie hoofdstuk 3).

6. UITLEG VAN `assets/css/styles.css`

6.1 CSS-VARIABELEN

In `:root` staan herbruikbare variabelen:

- 🟡 `--green`: hoofdgroen van de organisatie.
- 🟡 `--green-dark`: donkere groene tekst en hoverkleur.
- 🟡 `--blue`: hoofdblaauw.
- 🟡 `--blue-soft`: lichte blauwe achtergrond.
- 🟡 `--mint`: lichte groene achtergrond.
- 🟡 `--ink`: donkere tekstkleur.
- 🟡 `--muted`: grijze ondersteunende tekst.
- 🟡 `--line`: randkleur.
- 🟡 `--paper` en `--paper-soft`: achtergrond van kaarten, invoervelden en tabelkoppen.
- 🟡 `--danger`, `--error-*` en `--warning-*`: kleuren voor verwijderknoppen, foutmeldingen en de bijbestellijst.
- 🟡 `--shadow`: standaard schaduw.

Als het organisatiepalet later verandert, kunnen de kleuren centraal worden aangepast.

Licht en donker thema: alle kleuren in de stylesheet komen uit deze variabelen. `:root` bevat de lichte kleuren, `:root[data-theme="dark"]` dezelfde variabelen met donkere waarden. De class `ThemeManager` zet `data-theme="light"` of `data-theme="dark"` op het `<html>`-element, en de hele website verandert dan mee. Een klein script in de `<head>` van `index.html` zet het donkere thema al vóór het tekenen van de pagina, zodat de pagina niet eerst wit oplicht. Bij `Auto` gebruikt het script het thema van het vorige bezoek, dat `ThemeManager` bewaart onder `tvb-theme-last`. Alleen de witte tekst op groene knoppen, de letters in de avatars en de kleuren van de banner zijn in beide thema's hetzelfde.

De stylesheet is geordend van algemeen naar specifiek:

1. Globale variabelen en reset.
2. Basislayout en navigatie.
3. Hoofdcomponenten zoals hero, kaarten en medewerkerregels.
4. Admincomponenten, tabellen en meldingen.
5. Responsive regels voor kleinere schermen.

Herbruikbare stijlen staan gegroepeerd in componentselectors, bijvoorbeeld `.card`, `.primary-button`, `.stat-card` en `.admin-search`. Hierdoor blijft de styling centraal beheerd en hoeft dezelfde stijl niet op meerdere plekken te worden gekopieerd.

6.2 LAYOUT

De website gebruikt onder andere:

- 🟡 Flexbox voor navigatie en rijen.
- 🟡 CSS Grid voor de hoofdinhoud en statistieken.
- 🟡 De variabelen `--page-width` en `--gutter`, zodat bovenbalk, inhoud en footer dezelfde breedte en zijmarge hebben.
- 🟡 Een pagina die altijd minstens schermhoog is (`100dvh`), met de footer onderaan.

Media queries:

Media query	Wat er verandert
<code>max-width: 1100px</code>	Kleine teksten worden groter, zodat ze op een tablet of telefoon op armlengte leesbaar zijn
<code>min-width: 721px</code> en <code>max-width: 900px</code>	Kleinere zijmarge; de totalen komen onder de medewerkerlijst
<code>min-width: 721px</code> , <code>max-width: 1100px</code> en staand	Compactere banner op een staande tablet, zodat de lijst hoger begint
<code>max-width: 720px</code>	Telefoonindeling: alles onder elkaar, kleinere zijmarge, minder tekst in de bovenbalk
<code>max-width: 520px</code>	De naam naast het logo vervalt
<code>hover: none</code>	De hint <code>Ctrl K</code> verdwijnt op aanraakschermen zonder toetsenbord
<code>prefers-reduced-motion: reduce</code>	Geen animaties voor wie "minder beweging" heeft ingesteld

6.3 COMPONENTEN

De CSS bevat opmaak voor:

- De topbar, het logo en de themakeuze.
- De hero-banner.
- Kaarten, statistieken en de tipkaart.
- Medewerkerregels, het zoekveld en het bedrijfsfilter.
- Het productvenster met -- en +-knoppen, de begroeting en het voorstel `zelfde als vorige keer`.
- Het venster van de demo gezichtsherkenning met het ovale kader.
- Admin-modal, tabbladen, tabellen, filters en de voorraadlabels.
- Formulieren met foutmeldingen onder het veld.
- Lege toestanden (icoon, titel, uitleg en eventueel een knop).
- Toastmeldingen (succes met vinkje, fout met waarschuwingsteken).
- De SVG-iconen.
- Mobiele schermen en het donkere thema.

7. UITLEG VAN DE JAVASCRIPT (assets/js/).

7.1 OPSLAG EN VOORBEELDDATA

```
// config.js
export const STORAGE_KEY = "tvb-blikjesregistratie";
```

Dit is de naam waaronder de applicatie haar gegevens in `localStorage` bewaart.

Bij de eerste start worden acht voorbeeldmedewerkers aangemaakt, met de standaardproducten, de 13 bedrijven en één consumptiepunt (Hoofdkantoor bij TVB). Daarna worden de gegevens uit `localStorage` gebruikt.

De status heeft deze structuur (de velden staan uitgelegd in hoofdstuk 8):

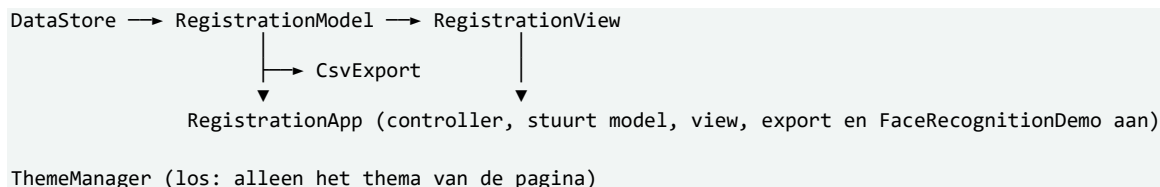
```
{
  employees: [
    {
      id: "unieke-id",
      name: "Lotte van Dijk",
      firstName: "Lotte",
      lastName: "van Dijk",
      payrollCode: "",
      personnelNumber: "",
      employerNumber: "",
      companyId: "id-van-bedrijf",
      pointId: "id-van-consumptiepunt",
      active: true,
      color: "#d8f1e8"
    }
  ],
  registrations: [
    {
      id: "unieke-id",
      employeeId: "id-van-medewerker",
      productId: "blikje",
      price: 0.65,
      pointId: "id-van-consumptiepunt",
      createdAt: "2026-10-01T10:05:00.000Z"
    }
  ],
  products: [{ id: "blikje", name: "Blikje", price: 0.65 }],
  companies: [{ id: "id-van-bedrijf", name: "TVB", employerNumber: "" }],
  points: [
    {
      id: "id-van-consumptiepunt",
      name: "Hoofdkantoor",
      companyId: "id-van-bedrijf",
      products: {
        blikje: { offered: true, stock: 24, minimum: 6, countedAt: "2026-10-01T07:30:00.000Z" }
      }
    }
  ],
  auditLog: [
    { id: "unieke-id", action: "Product gewijzigd", details: "Blikje", createdAt: "2026-10-01T10:06:00.000Z" }
  ]
}
```

De ingestelde gezichten van de demo gezichtsherkenning staan **niet** in deze status en ook niet in `localStorage`; ze bestaan alleen in het geheugen van de pagina.

Naast de gegevens zelf kunnen er twee reservekopieën in `localStorage` staan: `tvb-blikjesregistratie-backup` en `tvb-blikjesregistratie-backup-laatste` (Zie DataStore hieronder). De thema-instelling staat los daarvan onder `tvb-theme` en `tvb-theme-last`.

7.2 OBJECTGEORIËNTEERDE STRUCTUUR

De JavaScript-code volgt het principe van objectgeoriënteerd programmeren (OOP) en het MVC-patroon (Model, View, Controller). De code is opgedeeld in zeven classes: vijf voor de applicatie zelf, ThemeManager voor het lichte en donkere thema en FaceRecognitionDemo voor de demo gezichtsherkenning. Iedere class heeft één duidelijke verantwoordelijkheid, en bovenaan ieder bestand staat in een comment waarvoor de class is en met welke andere classes hij verbonden is.



main.js maakt alle objecten één keer aan en geeft ze aan elkaar door (compositie). config.js, icons.js en ids.js bevatten alleen vaste waarden of een hulpfunctie en zijn daarom geen class.

Toegepaste OOP-principes:

- 🔹 **Encapsulatie:** iedere class beheert zijn eigen gegevens. Andere classes lezen de gegevens van het model via getters (model.employees, model.products) en wijzigen ze alleen via methodes (model.addRegistration, model.saveCompany).
- 🔹 **Eén verantwoordelijkheid per class:** opslag, regels, weergave, besturing en export staan elk in een eigen class.
- 🔹 **Compositie:** de controller krijgt het model, de view en de export mee in de constructor, in plaats van ze zelf te maken. Daardoor kan in de tests een nep-view of nep-export worden meegegeven.
- 🔹 **Losse koppeling:** de view weet niets van de controller en verandert geen gegevens. De opslag kan worden vervangen door een andere class met dezelfde methodes load() en save().

7.2.1 DataStore

Verzorgt het lezen en opslaan van gegevens in localStorage. load leest de gegevens, zet ze om en slaat losse beschadigde regels over (removeInvalidItems). Zijn er regels overgeslagen of zijn de gegevens helemaal onleesbaar, dan bewaart keepBackup de oorspronkelijke gegevens als reservekopie en zet loadProblem op "skipped" of "unreadable"; de controller toont dan een melding. Er zijn hooguit twee reservekopieën (backupKeys): tvb-blikjesregistratie-backup is de eerste en wordt nooit overschreven, tvb-blikjesregistratie-backup-laatste is de nieuwste en wordt steeds vervangen. Staat dezelfde kopie er al als eerste, dan gebeurt er niets. Zo loopt de opslag niet vol.

Bij het laden wordt iedere regel gecontroleerd. isSafeId eist dat iedere id en iedere verwijzing naar een id alleen letters, cijfers, - en _ bevat (maximaal 64 tekens, ^[A-Za-z0-9_-]{1,64}\$). Zo kan een aangepaste id niet uit een HTML-attribuut breken. isSafeColor eist een kleur in de vorm #rrggbb, en isValidPrice een prijs van 0 tot en met 1000. Een regel met een ongeldige id, verwijzing of prijs wordt overgeslagen; een product zonder prijs (null of leeg) ook, zodat het niet gratis wordt. Komt een id binnen één lijst (medewerkers, registraties, producten, bedrijven, consumptiepunten of logboek) meer dan één keer voor, dan blijft alleen de eerste regel staan (withoutDuplicateIds). Een registratie met een tijdstip (createdAt) van meer dan 24 uur na het moment van laden wordt ook overgeslagen. Net als bij andere overgeslagen regels komen er dan een reservekopie, loadProblem "skipped" en een melding. Een medewerker met een ongeldige kleur blijft bewaard en krijgt een kleur uit COLORS.

`clearAll` wist de gegevens, beide reservekopieën en oude reservekopieën met een tijdstempel uit een eerdere versie. Dit wordt gebruikt door de knop `Alle gegevens wissen`.

`migrate` zet oude gegevens om (bijvoorbeeld product-id `melk` naar `glas-melk` en `brood` naar `sneetje-brood`, in de registraties, in de opgeslagen productlijst en in de voorraad van de punten, of een vrij ingevulde bedrijfsnaam naar een echt bedrijf). Opgeslagen producten gaan altijd voor op de standaardproducten, zodat gewijzigde prijzen en verwijderde producten bewaard blijven. Voorraad en minimum worden altijd getallen (een negatief minimum wordt 0; een negatieve voorraad blijft staan), het tijdstip van de laatste telling (`countedAt`) blijft bewaard als het een geldige datum is die niet in de toekomst ligt (een telling in de toekomst wordt weggelaten, zodat de voorraad niet geblokkeerd raakt; het product telt dan als "niet geteld"), en beschadigde logboekregels (`isValidAuditEntry`) worden overgeslagen, zodat het logboek niet vastloopt. Als later een database wordt gebruikt, kan deze opslaglaag worden vervangen zonder de rest opnieuw te schrijven.

7.2.2 `RegistrationModel`

Bevat alle gegevens en regels. De methodes zijn gegroepeerd per onderwerp: registraties, medewerkers, producten, bedrijven, consumptiepunten, voorraad en logboek. Voorbeelden van regels:

- Een registratie verlaagt de voorraad van het consumptiepunt van de medewerker, als dat punt het product aanbiedt en de registratie niet vóór de laatste telling ligt. `pointId` in de registratie is het punt dat het product aanbood (anders `null`).
- Een bedrijf of consumptiepunt met medewerkers kan niet worden verwijderd.
- `removeEmployee` verwijdert alleen een inactieve medewerker zonder registraties; anders geeft het `false`.
- `saveCompany` weigert (`false`) een werkgevernnummer dat al bij een ander bedrijf hoort (`isEmployerNumberTaken`); een leeg nummer telt niet.
- De loonmaand is de maand na de consumptie (`payrollPeriod`).
- Met `snapshot` en `restore` kan een wijziging worden teruggedraaid als opslaan mislukt.
- `lastRegistration` zoekt de laatst ingevoerde registratie van een medewerker (en product): de volgorde van invoer telt, niet de datum. Dat is precies de registratie die `removeLastRegistration` zou verwijderen, zodat de controller vooraf kan vragen of een registratie uit een oude maand echt weg mag.
- `setStock` vervangt de voorraad door de getelde hoeveelheid en bewaart het tijdstip (`countedAt`). `removeLastRegistration` zet een product alleen terug in de voorraad als de registratie na die telling is gemaakt. `stockEntry` geeft de voorraadregel van één product op één punt.
- `moveEmployeesOfPoint` zet de medewerkers van een consumptiepunt over naar een ander bedrijf. Alleen `companyId` verandert; de naam wordt niet opnieuw opgebouwd.
- `wipeAll` laat de opslag alles wissen (`clearAll`) en start opnieuw met de demogegevens (`createSeedState`).

7.2.3 `RegistrationView`

Verzorgt alleen de presentatie in de browser. Deze class rendert de medewerkerlijst, het productvenster, statistieken, admin-tabellen, filters, voorraad en toastmeldingen. Alle tekst én alle waarden in HTML-attributen (zoals id's in `data-...`, `value=..."` en klassen) gaan eerst door `escapeHtml`, zodat ze niet uit het attribuut kunnen breken. De functie `cssAttributeValue` (in hetzelfde bestand, ook gebruikt door de controller) maakt een id veilig voordat die in een CSS-selector komt; in de browser gebruikt die `CSS.escape`. `companyGroupKey` bepaalt bij welke groep van het bedrijfsfilter een medewerker hoort: medewerkers zonder bedrijf of met een bedrijf dat niet meer bestaat, vallen onder Onbekend bedrijf. `populateCompanyFilter` overschrijft de tekst

in het bedrijfsfilter niet zolang de gebruiker erin typt. `countRegistrations` telt de registraties per medewerker en product in één keer, zodat de correctielijst ook bij veel registraties snel blijft. `renderStockAlerts` en `updateStockStatus` werken na het aanpassen van de voorraad alleen de bijbestellijst en één statuslabel bij. De view leest uit het model, maar verandert zelf niets.

7.2.4 RegistrationApp

Vormt de controller van de applicatie. Deze class koppelt klik- en formulier-events aan het model en laat daarna de view opnieuw renderen. `handleClick` bevat een lijst van knoppen met de actie die erbij hoort; iedere actie is een eigen, korte methode. De methode `persist(wijziging, logboekregel)` zorgt dat iedere wijziging wordt opgeslagen of automatisch wordt teruggedraaid bij een opslagfout. De logboekregel (bijvoorbeeld `{ action: "Product verwijderd", details: "Blikje" }`) wordt toegevoegd vóórdat er één keer wordt opgeslagen. Zo worden de wijziging en de logboekregel samen opgeslagen of samen teruggedraaid. `finishAdminChange` tekent daarna alle beheeronderdelen opnieuw, ook de registratietabel (zodat bijvoorbeeld een nieuwe productnaam daar meteen staat), zonder nog eens apart op te slaan.

`handleStorageChange` luistert naar het `storage`-event: slaat een ander tabblad iets op, dan laadt het model de gegevens opnieuw (`RegistrationModel.reload`) en wordt alles opnieuw getekend. Daarna ruimt de controller op wat niet meer klopt: `closeStaleForms` sluit een open wijzigformulier van iets dat intussen is verwijderd, en een open productvenster (met het cameravenster erboven) sluit met een melding als de medewerker is verwijderd of gedeactiveerd. Typt de beheerder in de voorraadtabel (`isTypingInStockTable`), dan blijft die tabel staan. Waren de nieuwe gegevens onleesbaar, dan volgt een waarschuwing. Klikt iemand in een ander tabblad op `Alle gegevens wissen`, dan verwijdert dat tabblad de gegevenssleutel (`event.newValue` is dan `null`). Dit tabblad wist dan ook alles uit het geheugen, logt de beheerder uit, vergeet de gezichten en toont "De gegevens zijn in een ander venster gewist." (`handleStorageCleared({ keepStore: true })`). De opslag zelf blijft met rust, want het andere tabblad zet er meteen de nieuwe demogegevens in; anders zouden de twee tabbladen elkaar steeds opnieuw wissen. Knoppen en formulieren van iets dat niet meer bestaat, geven via `showMissing` een melding in plaats van het gegeven opnieuw aan te maken.

Andere methodes:

- 🔹 `refreshIfNewDay`: een timer roept deze methode iedere minuut aan. Is de kalenderdag veranderd, dan worden de datum (`renderTodayLabel`) en de totalen (`renderStats`) opnieuw getekend.
- 🔹 `openModal` en `closeModal`: openen en sluiten een venster. `openModal` onthoudt welk element de focus had (`rememberOpener`); `closeModal` zet de focus daar weer op, als dat element nog zichtbaar op de pagina staat.
- 🔹 `confirmOldMonthRemoval`: vraagt bevestiging voordat een correctie met `-` een registratie uit een andere maand verwijdert.
- 🔹 `setSaveButtons`: zet de opslagknoppen van het medewerkers- en productformulier goed. Bij toevoegen staan er `Opslaan` (secundair) en `Opslaan + opnieuw` (primair); bij wijzigen alleen `Opslaan`, en die is dan primair.
- 🔹 `resetCorrectionDate`, `readCorrectionDate` en `confirmOldMonthAddition`: het datumveld bij de correctie `+`. Na het inloggen staat het op vandaag; bij `+` wordt de datum gecontroleerd (niet in de toekomst, niet vóór 2000) en volgt bij een eerdere maand een bevestigingsvraag.
- 🔹 `saveStockField` en `bookDelivery`: controleren dat voorraad en minimum hele getallen van 0 tot en met 100.000 zijn, en een levering van 1 tot en met 100.000. `isRepeatedDeliveryTap` negeert een tweede tik op dezelfde knop `Toevoegen` binnen 800 milliseconden.

- 🔗 `wipeAllData`: de knop Alle gegevens wissen. Doet niets als er geen beheerder is ingelogd. Vraagt twee keer om bevestiging, laat het model alles wissen (`wipeAll`), slaat de nieuwe demogegevens op, vergeet de ingestelde gezichten, logt de beheerder uit en tekent alles opnieuw.

7.2.5 `CsvExport`

Maakt het CSV-bestand voor de loonadministratie (zie verderop).

7.2.6 `ThemeManager`

Regelt het lichte en donkere thema (zie 4.13). Onthoudt de instelling (`system`, `auto`, `light` of `dark`) in `localStorage`, bepaalt met `resolveTheme` welk thema daarbij hoort en zet dat met `apply` op de pagina. Deze class staat los van het model, de view en de controller, omdat het thema een weergave-instelling per browser is en geen gegeven van de registratie.

7.2.7 `FaceRecognitionDemo`

De uitschakelbare demo gezichtsherkenning (zie 4.15). De class laadt de bibliotheek `face-api` pas als iemand de demo gebruikt (`load`), uit `assets/vendor/face-api/` in het project zelf, zet de camera aan en uit (`startCamera`, `stopCamera`; is het venster gesloten terwijl de camera startte, dan zet `startCamera` die camera meteen weer uit) en leest het grootste gezicht in beeld (`readFace`, `largestFace`). Een gezicht wordt bewaard als een rij getallen (een descriptor), alleen in het geheugen (`enroll`, `forget`). `findMatch` zoekt het ingestelde gezicht met de kleinste afstand onder de drempel uit `FACE_DEMO.matchThreshold`, en `confirm` telt pas als dezelfde persoon twee keer achter elkaar is gevonden. De controller krijgt deze class mee in de constructor. Mislukt het afspelen van het camerabeeld nadat de camera al aan stond, dan zet de controller de camera uit. Of er een gezicht wordt vastgelegd, hoort bij één keer openen van het cameravenster (`captureSession`); een vastlegging uit een gesloten venster blokkeert een opnieuw geopend venster dus niet. De camera-functie van de browser (`mediaDevices`) kan worden meegegeven, zodat de camera in de unit tests kan worden nagebootst.

7.2.8 `icons.js`

Bevat de SVG-iconen als lijntekeningen met dezelfde lijndikte. De functie `icon(naam)` geeft de HTML van één icoon; elementen met `data-icon="naam"` in `index.html` krijgen bij het starten hun icoon.

7.3 `renderEmployees`

De methode `RegistrationView.renderEmployees`:

1. Leest de zoekterm en het gekozen bedrijf.
2. Toont alleen actieve medewerkers die daarbij passen.
3. Groepeert de medewerkers per bedrijf.
4. Maakt voor iedere medewerker een klikbare rij (ook te bedienen met Enter of spatie).

De `data-open-employee`-waarde bevat de unieke medewerker-id. Daardoor weet de klikhandler voor welke medewerker het productvenster moet worden geopend.

7.4 `renderStats`

De methode `RegistrationView.renderStats` vergelijkt de datum van iedere registratie met:

- 🔗 De datum van vandaag.
- 🔗 De sleutel van de huidige maand.

Daarna worden de twee totalen in de statistiekkarten geplaatst. Na middernacht roept `RegistrationApp.refreshIfNewDay` deze methode opnieuw aan, zodat de totalen bij de nieuwe dag horen.

7.5 `registerSelectedProducts`

De methode `RegistrationApp.registerSelectedProducts` wordt uitgevoerd na een klik op Registreren in het productvenster. Voor ieder gekozen product maakt `RegistrationModel.addRegistration` een object aan met:

- Een unieke registratie-id.
- De medewerker-id.
- De product-id.
- De huidige datum en tijd in ISO-formaat.

Daarna worden de gegevens via `persist` opgeslagen en alle zichtbare onderdelen opnieuw getekend. Correcties van de beheerder met de +-knop lopen via `RegistrationApp.addCorrection` en komen ook in het logboek.

7.6 `renderAll`

De methode `RegistrationView.renderAll` voert `populateCompanyFilter`, `renderEmployees` en `renderStats` achter elkaar uit. Zo blijven het bedrijfsfilter, de medewerkerlijst en de totalen gelijktijdig actueel.

7.7 `renderEmployeeProducts`

De methode `RegistrationView.renderEmployeeProducts` tekent het productvenster: de begroeting (`greeting` kiest Goedemorgen, Goedemiddag of Goedenavond), het voorstel `zelfde als vorige` keer (uit `RegistrationModel.lastSelection`, beschreven met `describeSelection`), de producten van het consumptiepunt met -- en +-knoppen en onderaan de link of status van de demo gezichtsherkenning.

7.8 `showToast`

De methode `RegistrationView.showToast` toont een korte melding onderaan in het midden van het scherm, zoals Lotte van Dijk: 2x Blikje, 1x Ei geregistreerd met een vinkje ervoor. Foutmeldingen krijgen een waarschuwingsteken. Standaard verdwijnt een melding na 2,5 seconde; de melding na het registreren blijft 4 seconden staan, zodat een medewerker aan een gedeelte tablet kan lezen voor wie en wat er is opgeslagen.

7.9 OPSLAGFOUTEN EN TERUGDRAAIEN

Iedere wijziging loopt via `RegistrationApp.persist`. Eerst wordt een snapshot van de vorige status gemaakt. Daarna wordt de wijziging uitgevoerd, wordt (bij een beheeractie) de logboekregel toegevoegd en probeert `DataStore.save` alles in één keer op te slaan. Als de browseropslag mislukt, wordt de vorige status teruggezet, dus zonder de wijziging én zonder de logboekregel, en krijgt de gebruiker een foutmelding. Zo lijkt een mislukte opslag niet toch succesvol, en staat er nooit een wijziging zonder logboekregel (of andersom).

7.10 ADMINFUNCTIES

`openAdmin` opent de modal. Als `adminLoggedIn` waar is, wordt het dashboard opnieuw getoond; anders verschijnt het loginformulier.

De variabele `adminLoggedIn` houdt de adminstatus alleen bij zolang de huidige pagina open is. `closeAdmin` (sluiten van het venster) en `logout` (knop Uitloggen) zetten `adminLoggedIn` op false en maken het wachtwoordveld leeg.

`renderAdmin`:

- Leest de gekozen medewerkerfilter.
- Leest de gekozen maandfilter.
- Filttert de registraties.
- Toont de resultaten in de admin-tabel.

`populateFilters` vult de dropdowns met de huidige medewerkers en maanden die in registraties voorkomen.

`renderAdminEmployees` toont alle medewerkers in het beheertabblad.

7.11 CSV-EXPORT (CLASS `CsvExport` IN `csvExport.js`)

De export heeft geen externe bibliotheek nodig. Na een klik op `CSV exporteren` roept `RegistrationApp.exportCsv` (alleen als een beheerder is ingelogd) de methode `CsvExport.download` aan:

- `buildRows` verzamelt de registraties die bij de filters passen en telt ze per medewerker per loonmaand.
- `RegistrationModel.payrollPeriod` bepaalt de loonmaand: de maand na de consumptie (december gaat naar januari van het volgende jaar).
- `toCsv` zet de regels om naar CSV met puntkomma's. `escapeField` zet velden met speciale tekens tussen aanhalingstekens, en geeft waarden die met `=`, `+`, `-` of `@` beginnen een `'` ervoor, zodat Excel ze niet als formule uitvoert. Dat geldt ook als er eerst spaties, tabs of enters voor dat teken staan (zoals `" =1+1"`). Een waarde die met een tab, `\r` of `\n` begint, krijgt altijd een `'` ervoor, ook zonder formuleteken. Let op: ook een gewone waarde die met `-` begint, krijgt zo een `'` ervoor.
- `fileName` maakt de bestandsnaam met de loonmaand.
- Het bestand wordt gedownload als `.csv` (UTF-8 met BOM, zodat Excel letters zoals `é` goed toont).

Zijn er geen registraties voor de filters, dan geeft `download` `false` terug en toont de controller een melding.

7.12 EVENT LISTENERS

De event listeners koppelen gebruikersacties aan functies:

De listeners worden in `RegistrationApp` per onderwerp gekoppeld:

Methode	Wat er wordt gekoppeld
<code>registerPageEvents</code>	De algemene kliklistener (<code>handleClick</code>), het sluiten van het productvenster en het cameravenster, de knoppen Herken mij met de camera (demo) en Gezicht vastleggen, en het toestemmingsvinkje
<code>registerAdminEvents</code>	Admin login, het sluiten van het admin-venster (dat ook uitlogt), inloggen, uitloggen, de tabbladen, Meer laden, CSV exporteren en Alle gegevens wissen
<code>registerFilterEvents</code>	De zoekbalk, het bedrijfsfilter, de filters Medewerker en Maand en de zoekvelden in het beheer
<code>registerFormEvents</code>	Openen, opslaan en sluiten van de formulieren voor medewerker, product, bedrijf en consumptiepunt, en het weghalen van een foutmelding zodra een veld wordt aangepast

Methode	Wat er wordt gekoppeld
registerStockEvents	De keuze van het consumptiepunt en het aanpassen van voorraad of minimum
registerKeyboardEvents	Escape sluit het bovenste venster (<code>topModal</code>), Tab blijft binnen een open venster (<code>keepFocusInModal</code>), Ctrl/Cmd + K voor de zoekbalk (alleen zonder open venster), en Enter of spatie op een medewerkerrij

Voor dynamisch aangemaakte knoppen gebruikt de app één algemene kliklistener op `document` (`handleClick`). Die bevat een lijst van `data--`attributen met de actie die erbij hoort, bijvoorbeeld `data-open-employee`, `data-product-increment`, `data-repeat-last` en `data-face-enroll`. Daardoor blijven knoppen werken nadat een lijst opnieuw is opgebouwd.

8. DATAMODEL EN KOPPELINGEN.

De huidige demo gebruikt JavaScript-objecten. De logische koppeling is:

8.1 BEDRIJVEN

Veld	Betekenis
id	Unieke identificatie
name	Bedrijfsnaam
employerNumber	Standaard werkgevernnummer voor de CSV-export

8.2 CONSUMPTIEPUNTEN

Veld	Betekenis
id	Unieke identificatie
name	Naam van het punt, bijv. Kantine begane grond
companyId	Verwijzing naar het bedrijf
products	Per product-id: offered (aangeboden ja/nee), stock (voorraad), minimum en, als er is geteld, countedAt (tijdstip van de laatste telling)

8.3 MEDEWERKERS

Veld	Betekenis
id	Unieke identificatie
name, firstName, lastName	Naam
payrollCode, personnelNumber	Looncode en personeelsnummer
employerNumber	Optioneel afwijkend werkgevernnummer; leeg = dat van het bedrijf (RegistrationModel.employerNumberFor)
companyId	Verwijzing naar het bedrijf
pointId	Verwijzing naar het vaste consumptiepunt
active	Actief of inactief
color	Kleur van de avatar

8.4 REGISTRATIES

Veld	Betekenis
id	Unieke identificatie
employeeId	Verwijzing naar een medewerker
productId	Verwijzing naar een product
price	Prijs op het moment van registreren (voor de CSV-export)
employerNumber	Werkgevernnummer op het moment van registreren (voor de CSV-export, zie RegistrationModel.registrationEmployerNumber). Ontbreekt als er toen nog geen nummer was; dan gebruikt de export het huidige nummer
pointId	Consumptiepunt waar de voorraad van af ging; bij een correctie gaat het product hier weer naartoe terug. Leeg (null) als het punt het product niet aanbiedt en er dus geen voorraad af ging

Veld	Betekenis
correction	Alleen aanwezig (true) als de beheerder de registratie met + heeft toegevoegd
createdAt	Datum en tijd van registratie. Bij een correctie + met een eerdere datum is dat 12:00 uur op die dag

Oude gegevens uit een eerdere versie worden bij het laden automatisch omgezet (DataStore.migrateCompanies): de vrij ingevulde bedrijfsnaam wordt een bedrijf, het eerste werkgevern timer wordt de standaard van dat bedrijf (een afwijkend nummer blijft bij de medewerker staan) en ieder bedrijf met medewerkers krijgt één consumptiepunt met alle producten.

Het aantal is in de demo altijd 1, omdat iedere klik één registratie maakt. Dit sluit aan bij het voorgestelde datamodel waarin Aantal standaard 1 is.

Een medewerker met registraties kan niet definitief worden verwijderd (alleen gedeactiveerd), zodat de export altijd een naam en looncode heeft. Ontbreekt een medewerker toch (bijvoorbeeld in oude gegevens), dan wordt bij die registraties Verwijderd getoond.

Voor de CSV-export wordt de prijs gebruikt die bij iedere registratie is bewaard (price); oude registraties zonder prijs gebruiken de huidige prijs uit het productbeheer. Het bestand bevat per medewerker per loonmaand het aantal en de totale prijs. Deze gegevens zijn alleen zichtbaar voor de beheerder.

9. BELANGRIJKE DEMO-BEPERKINGEN.

Deze versie is bedoeld als prototype:

- ⬢ `localStorage` is niet gedeeld tussen gebruikers of apparaten. Tussen tabbladen in dezelfde browser wel: slaat het ene tabblad iets op, dan laadt het andere tabblad de nieuwe gegevens in.
- ⬢ Zijn de opgeslagen gegevens beschadigd, dan worden losse kapotte regels overgeslagen (of start de demo opnieuw als alles onleesbaar is). De oorspronkelijke gegevens staan dan als reservekopie in `localStorage` (`tvb-blikjesregistratie-backup`, en bij een volgende keer `tvb-blikjesregistratie-backup-laatste`), en de gebruiker krijgt een melding.
- ⬢ Er is geen echte database.
- ⬢ De login heeft geen echte beveiliging: elk ingevuld wachtwoord werkt.
- ⬢ Alle gegevens staan als gewone tekst in `localStorage` van de tablet. Wie het apparaat en de ontwikkelaarstools van de browser gebruikt, kan ze lezen en aanpassen, en kan via de console ook de code van de pagina uitvoeren.
- ⬢ Er zijn geen gebruikersrollen of wachtwoordbeheer.
- ⬢ Verwijderde medewerkers krijgen geen herstelmogelijkheid, en Alle gegevens wissen kan niet ongedaan worden gemaakt.
- ⬢ Er is geen server-side auditlog; het logboek staat alleen in de browser.
- ⬢ Bescherming tegen inbedden in een frame (`frame-ancestors` of `X-Frame-Options`) kan alleen met een HTTP-header van de webserver en zit dus niet in de demo.
- ⬢ De gezichtsherkenning is alleen een demo. Ingestelde gezichten zijn na het herladen van de pagina weg. Er is geen controle of er een echt, levend gezicht voor de camera staat, en hoe vaak de verkeerde persoon wordt herkend, is niet gemeten. Bij het laden toont de console één onschadelijke CSP-melding over `wasm-eval`, omdat de bibliotheek test of WebAssembly werkt. Zie het TO (hoofdstuk 13) voor de privacyregels en de beveiliging.

Voor productie is een backend met een gedeelde database, echte authenticatie, autorisatie, back-ups en auditlogging nodig. Het TO (hoofdstuk 13) bevat de volledige lijst van wat vóór productie moet gebeuren.

10. CODE UITLEGGEN TIJDENS EEN PRESENTATIE.

De applicatie volgt het principe van separation of concerns:

1. `DataStore` weet hoe gegevens worden opgeslagen.
2. `RegistrationModel` weet welke gegevens en bedrijfsregels gelden.
3. `RegistrationView` weet hoe gegevens in HTML worden getoond.
4. `RegistrationApp` weet welke actie bij een klik of formulier hoort.
5. `CsvExport` weet hoe het CSV-bestand voor de loonadministratie eruitziet.
6. `ThemeManager` weet welk thema (licht of donker) actief moet zijn.
7. `FaceRecognitionDemo` weet hoe de camera en de demo gezichtsherkenning werken.

De schermen zelf staan als screenshots in het TO, hoofdstuk 9, paragraaf Wireframes ([docs/wireframes/](#)). Die zijn handig om tijdens een presentatie te laten zien welke code bij welk scherm hoort.

De HTML bevat de componenten en vaste ankerpunten met `id`-waarden. De CSS bepaalt uitsluitend de presentatie. JavaScript koppelt de componenten aan gedrag. Daardoor kan bijvoorbeeld de opslag worden vervangen door een database zonder de volledige gebruikersinterface opnieuw te bouwen.

11. MOGELIJKE VERVOLGSTAPPEN.

1. Kies een backend, bijvoorbeeld PHP, Node.js of een beheerde dienst.
2. Maak de tabellen uit `docs/DATABASE-SCHEMA.sql` aan (onder andere medewerkers, registraties, producten, bedrijven, consumptiepunten en het logboek).
`point_products.counted_at` bewaart het tijdstip van de laatste telling (zoals `countedAt` in de demo), en een beheerder heeft de rol `admin` (beheerder) of `system_admin` (systeembeheerder).
3. Vervang `localStorage` door API-aanroepen; alleen `DataStore` hoeft daarvoor te veranderen.
4. Voeg echte admin-authenticatie toe.
5. Bewaar het logboek op de server in plaats van in de browser.
6. Voeg naast het maandfilter ook jaar- en periodefilters toe.
7. Voeg databaseback-ups en foutlogging toe.
8. Test met meerdere gebruikers tegelijk.